LAB 7

Fuzzing for Input Validation Bugs

Property-based fuzz testing of an irrigation scheduler
with the Hypothesis library

Name	Gaurav Girish
SRN	PES1UG24AM103
Course	Software Engineering — Laboratory
Lab	Lab 7 — Fuzzing for Input Validation Bugs
Assigned repository	SETAPESU26/L6_L7_37_irrigation_scheduler
Module under test	irrigation_scheduler.py
Date	10 November 2026

Component	**Version**
macOS	Darwin 27.0.0 (arm64)
Python	3.13.0
pytest	9.2.0
hypothesis	6.170.15

1. Objective

To explore fuzz testing as a security and quality-assurance technique: to take a small Python module that parses user input, use the Hypothesis property-based fuzzing library to discover the input-validation bugs hiding in it, and then harden the module so that the same test suite passes unchanged.

The assigned repository is **L6_L7_37_irrigation_scheduler**. Its module exposes three functions that each parse a string the user types in:

Function	Input format	What it is meant to produce
`parse_zone()`	*name*`:`*area* e.g. north:120	A (title-cased zone name, area in m²) pair
`moisture_average()`	comma-separated whole percentages e.g. 40,35,50	The mean soil-moisture percentage
`water_needed()`	*area*`|`*rainfall* e.g. 120|8	Litres of water required, never below zero

A note on file names. The lab handout refers to the files generically as `processor.py` and `test_processor.py`. In the repository assigned to this SRN they are named `irrigation_scheduler.py` and `test_irrigation_scheduler.py`, and those are the names used throughout this report and in the submitted code. Every command shown is otherwise exactly the command given in the handout.

2. Fuzzing, and what counts as a bug here

A fuzzer feeds a program random, unexpected and malformed data in order to find the failures nobody thought to write a test for. A hand-written unit test checks what the author imagined might go wrong; a fuzzer explores what the author never imagined. Hypothesis is a *property-based* fuzzer: instead of listing individual cases, the test states a property that must hold for every input, and Hypothesis generates hundreds of inputs trying to falsify it. When it succeeds it *shrinks* the counter-example down to the smallest input that still fails, which is what makes the reports below so short and so easy to act on.

The property used throughout this lab is deliberately explicit. For every input Hypothesis can invent, each function must do exactly one of two things:

	Outcome	Meaning
(a)	**Accept** the input	Return a result that satisfies the function's documented post-conditions — correct type, finite, and inside the physical range the quantity can actually take.
(b)	**Reject** the input	Raise `InvalidInputError`, the module's single documented validation error, carrying a message the user can act on.

Anything else is a bug, and this splits the findings into two families. The first is the obvious one: an **uncontrolled crash**, where a `ValueError` escapes from `float()`, or an `AttributeError` from `None`, and the program dies with a traceback. The second is quieter and more dangerous: **silent acceptance of nonsense**. A soil-moisture reading of –5 %, a field area of –120 m², or an area of `inf` does not crash anything — it just produces an irrigation schedule that is quietly wrong. Asserting post-conditions, and not merely the absence of exceptions, is what lets the suite catch both.

3. Step 1 — Installing the dependencies

The two libraries required by the handout were installed with pip:

```
pip install hypothesis
pip install pytest
```

On this machine neither `python` nor a bare `pytest` is on the `PATH`, so every command below uses `python3` and the `python3 -m pytest` form that the handout gives as the fallback. The versions in use are the ones listed on the cover page.

4. Step 2 — Running the buggy application

The unmodified module was run first, to establish that it does work — as long as the input is exactly what the author expected. Figure 1 shows the three functions being exercised with well-formed input: a zone of 120 m², three moisture readings averaging 41 %, and 592 litres of water for 120 m² of ground that has had 8 mm of rain.

Figures 2 and 3 show how little it takes to break it. Omitting the `:` separator kills `parse_zone` with `ValueError: not enough values to unpack`, and a single doubled comma in the reading list kills `moisture_average` with `ValueError: invalid literal for int()`. In both cases the user is shown a raw Python traceback and the program exits.

The original module is kept in the `before/` folder of the submission so that every run in this section can be reproduced after the fix; that is why the shell prompt reads `before %` in Figures 1–7.

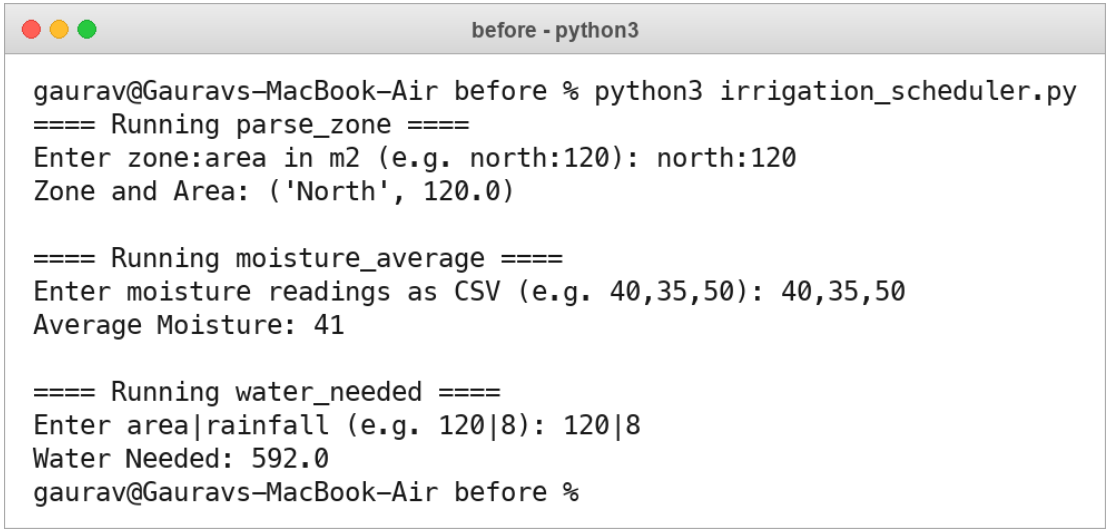


Figure 1 — Step 2: the unmodified module handles well-formed input correctly.

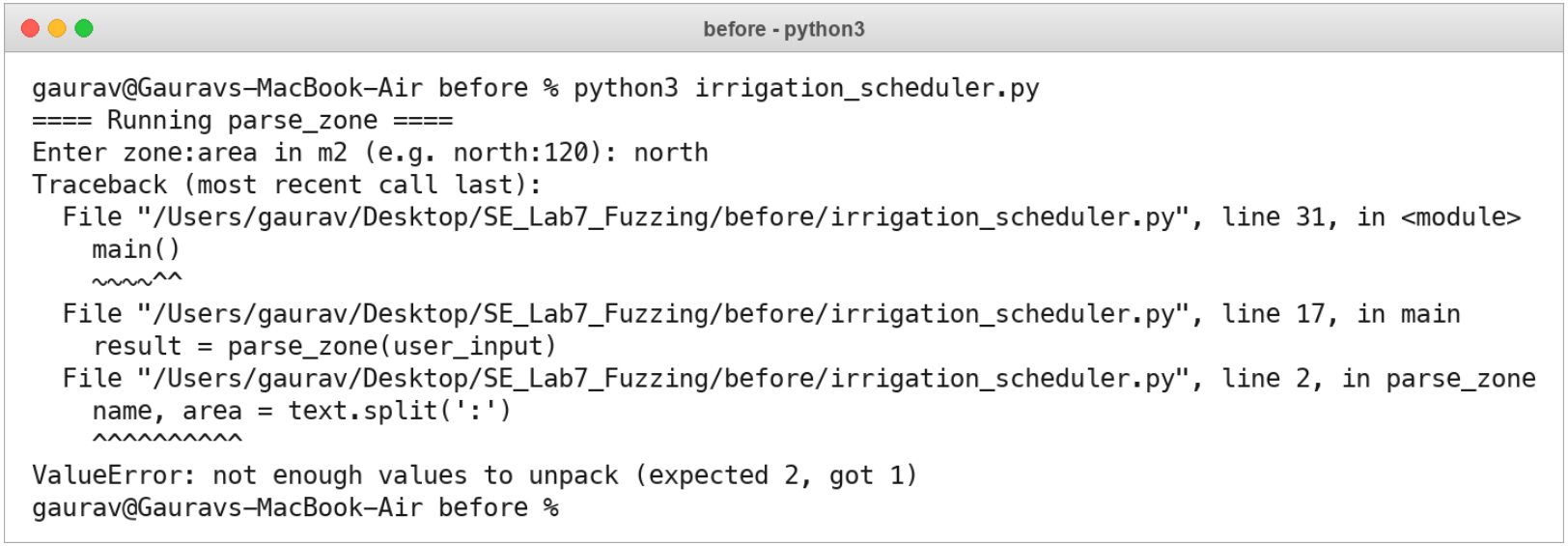


Figure 2 — Step 2: `parse_zone` crashes when the `:` separator is missing.

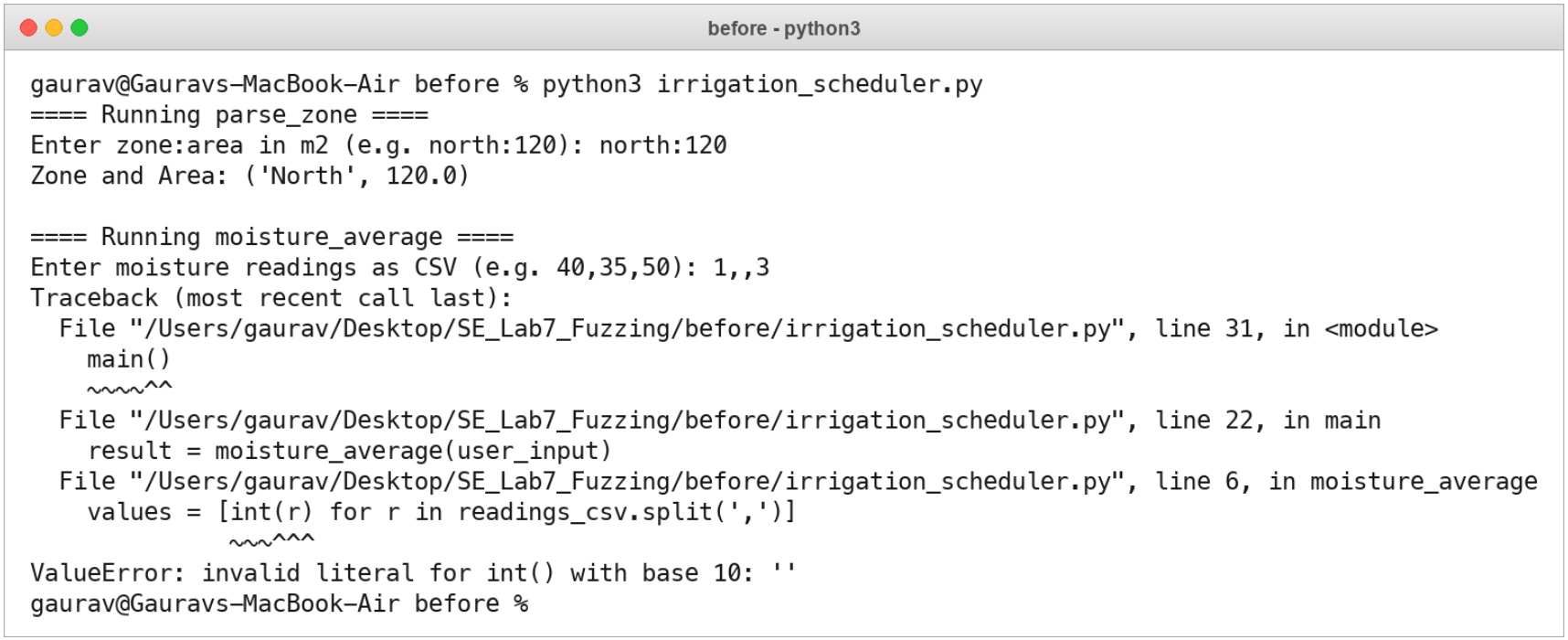


Figure 3 — Step 2: `moisture_average` crashes on an empty element in the CSV list (`1,,3`).

5. Step 3 — Completing the Hypothesis test suite

The starter file contained three decorated stubs with empty bodies. The completed suite keeps those three test names and the `st.text() | st.none()` idea behind them, and builds the rest of the suite around them. It is a single file of 59 tests in four groups.

5.1 Telling a rejection apart from a crash

Every fuzz test routes its call through one helper. The broad `except Exception` is not sloppiness — it is the bug detector, and it is the only place in the suite that catches anything:

```
def _call(func, raw):
    try:
        return "ok", func(raw)
    except InvalidInputError as exc:          # a deliberate rejection
        return "rejected", exc
    except Exception as exc:                  # anything else is a crash
        raise AssertionError(... ) from None
```

`InvalidInputError` does not exist in the buggy module, so the import of it falls back to a local placeholder class. This is what makes one unchanged file work on both sides of the fix: with the placeholder in place nothing counts as a deliberate rejection, so every malformed input is correctly reported as a crash and the suite fails. After Step 5 the real class is imported from the module and the same file passes.

5.2 The strategies

Unstructured fuzzing alone is not enough here, and it is worth being clear about why. Random text almost never contains a `:` or a `|`, so `st.text()` on its own trips over the very first `split()` every time and never reaches the `float()` and `int()` calls behind it. A second, structure-aware strategy glues suspicious fragments together with real separators so the fuzzer gets past the split and can attack the parsing and the arithmetic underneath.

Strategy	Purpose
`st.text() | st.none()`	The baseline from the handout: random Unicode, and the absent value.
`st.integers() | st.floats() | st.booleans() | st.binary() | st.lists(...) | st.dictionaries(...)`	Type confusion — the caller passes something that is not a string at all.
`st.sampled_from([...]) joined by st.sampled_from([':', '|', ',', ';', ' ', ''])`	Structure-aware fuzz: empty fields, `nan`, `inf`, `1e400`, negative numbers, percentages above 100, non-ASCII digits, doubled and wrong separators.
`st.from_regex(..., fullmatch=True)`	Well-formed input, used to prove the hardening did not change the answer for valid data.
`st.lists(st.integers(0, 100), min_size=1)`	Valid reading lists, compared against a reference implementation of the average.

5.3 The four groups of tests

Group	Tests	What it asserts
1. No-crash properties	3	The three names from the starter file. Each function either accepts the input and satisfies its post-conditions, or rejects it with `InvalidInputError`.
2. Behaviour on valid input	6	Hardening must not change the answer: round-trip of name and area, tolerance of surrounding whitespace, the average against a reference implementation, and the water estimate against the original formula.

Group	Tests	What it asserts
3. Regression tests	48	The concrete inputs Hypothesis shrank to, pinned down one per case so a regression names the exact value that broke.
4. Oversized input	2	A 100,000-element reading list and a 100,000-character zone name are refused rather than processed.

6. Step 4 — Running the fuzz tests against the buggy module

Against the unmodified module the suite reports **54 failed, 5 passed**. The five that pass are the Group 2 tests: the module does compute the right answer for well-formed input, which is worth knowing, because it means the 54 failures are all about input handling and not about the arithmetic. It also shows the suite is not trivially broken.

Figures 5 to 7 are the interesting ones. For each function Hypothesis reports several *distinct* failures and shrinks each to a minimal input — `None`, `''`, `b''`, `':'`. Nothing longer or more exotic is needed to crash this module.



Figure 4.1 — Step 4: the full suite against the buggy module — 54 failed, 5 passed.

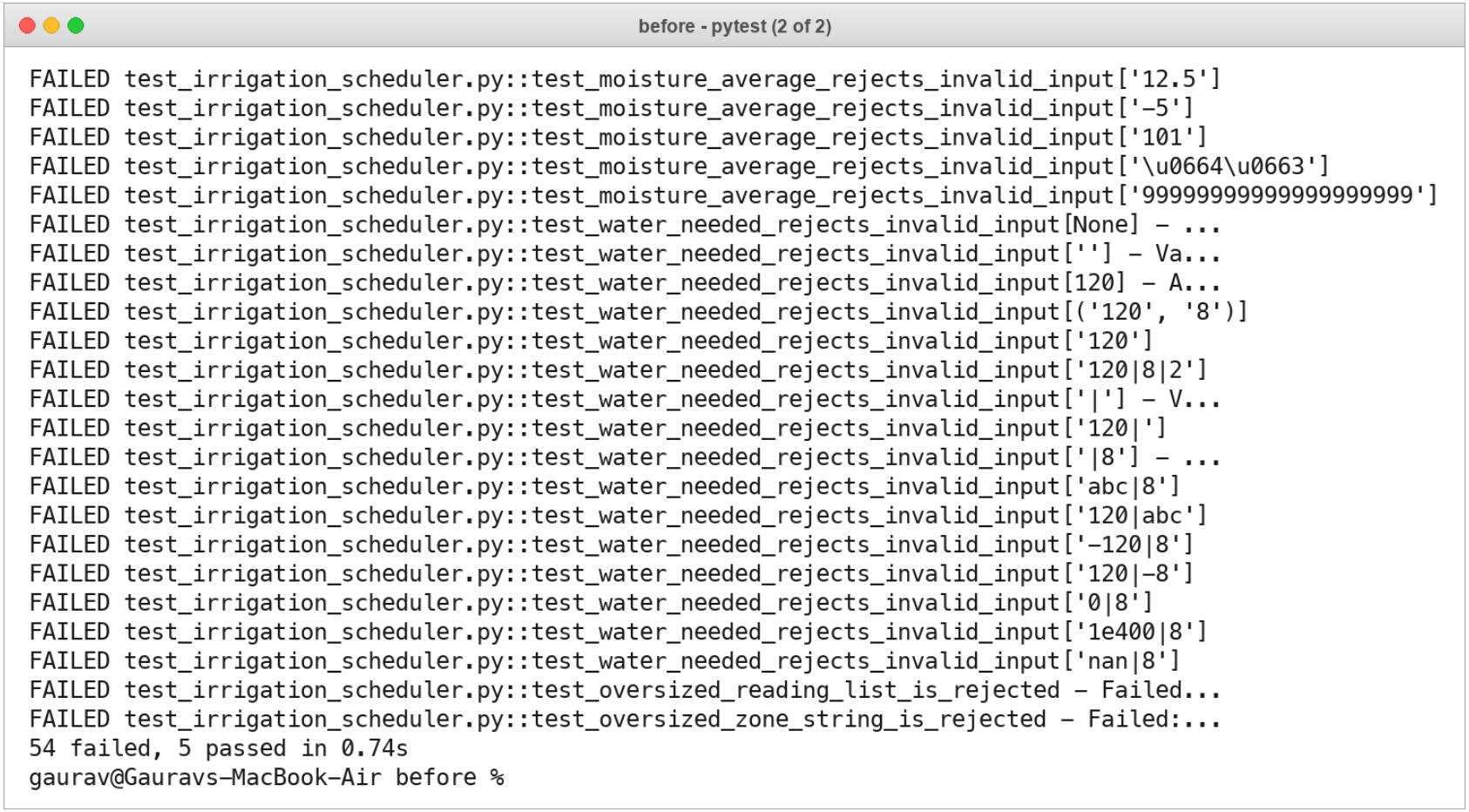


Figure 4.2 — Step 4: the full suite against the buggy module — 54 failed, 5 passed.

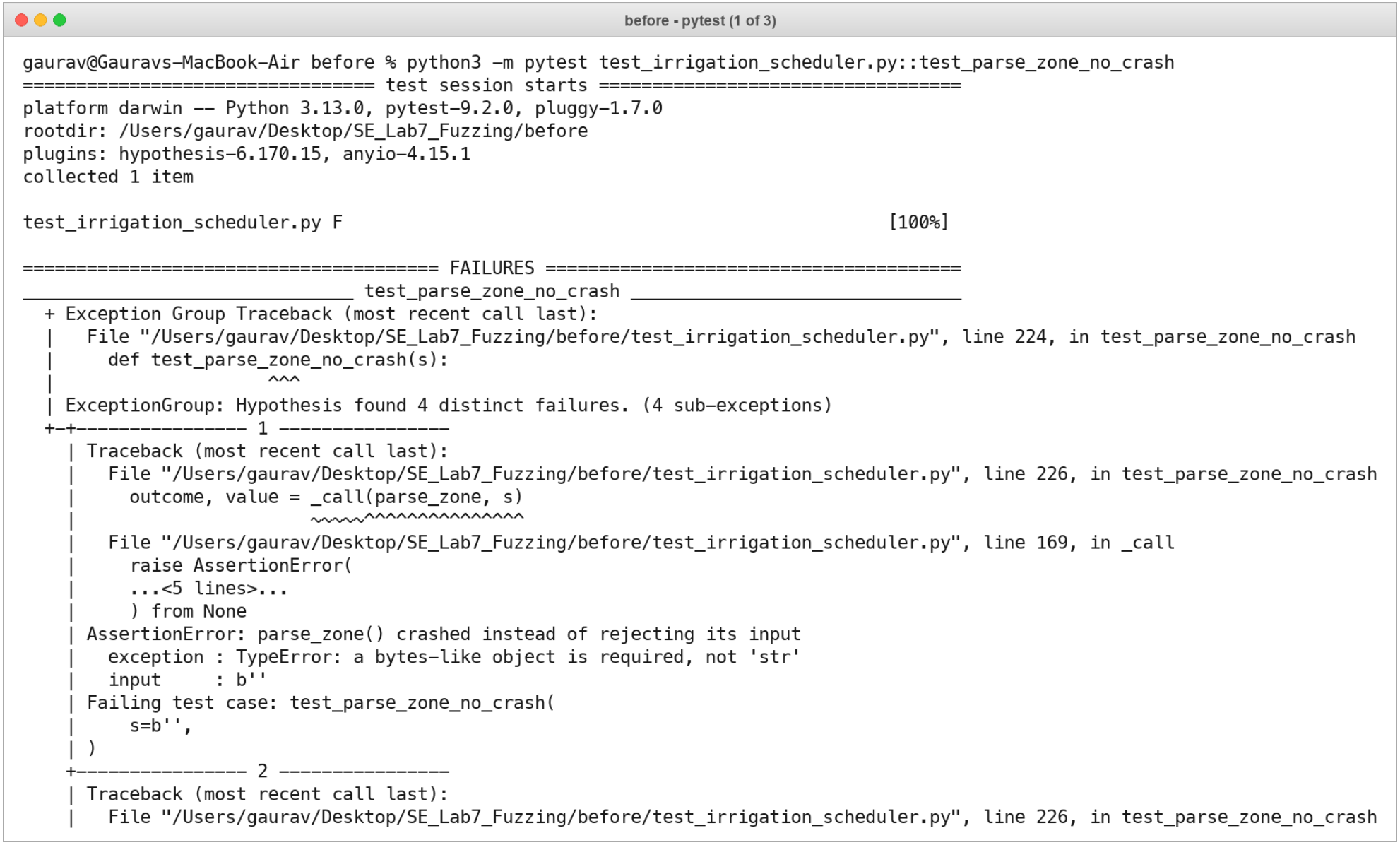


Figure 5.1 — Step 4: four distinct crashes in `parse_zone`, each shrunk to a minimal input.

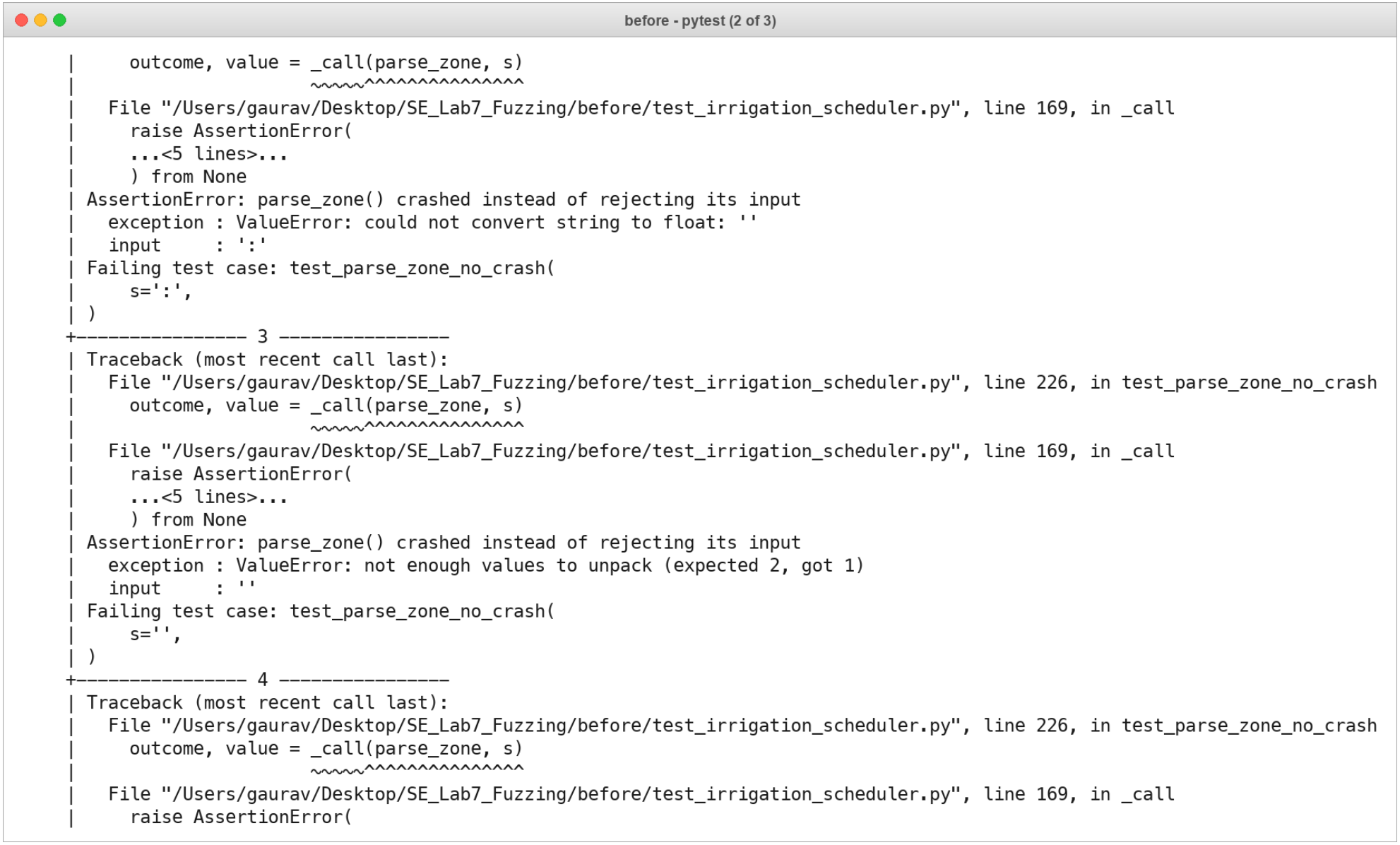


Figure 5.2 — Step 4: four distinct crashes in `parse_zone`, each shrunk to a minimal input.

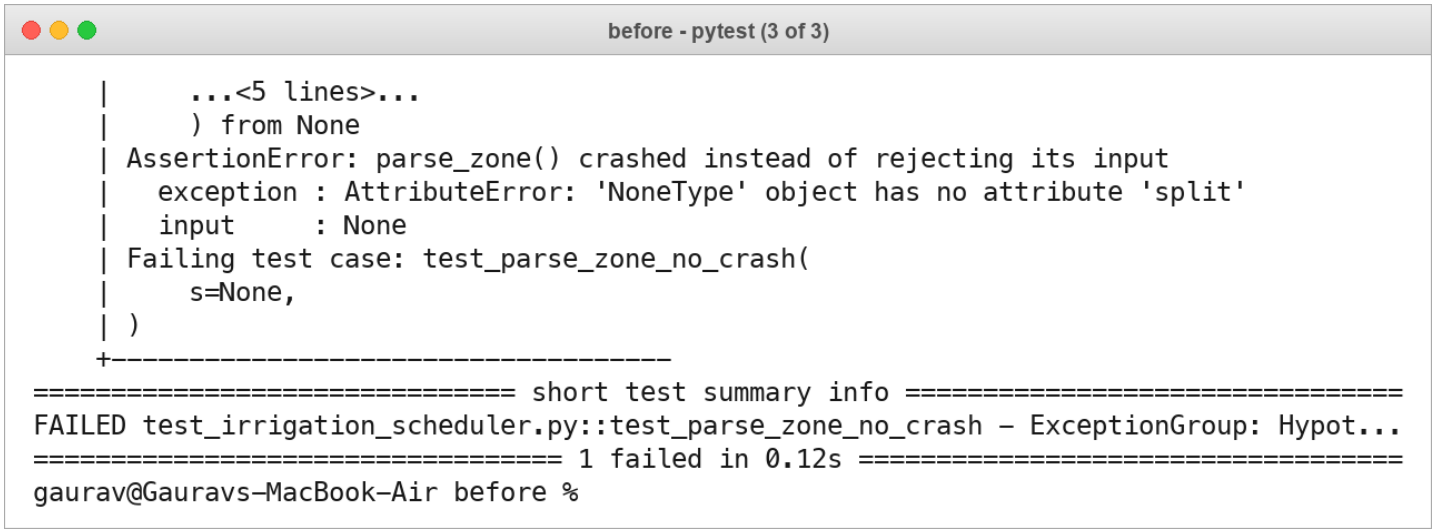


Figure 5.3 — Step 4: four distinct crashes in `parse_zone`, each shrunk to a minimal input.

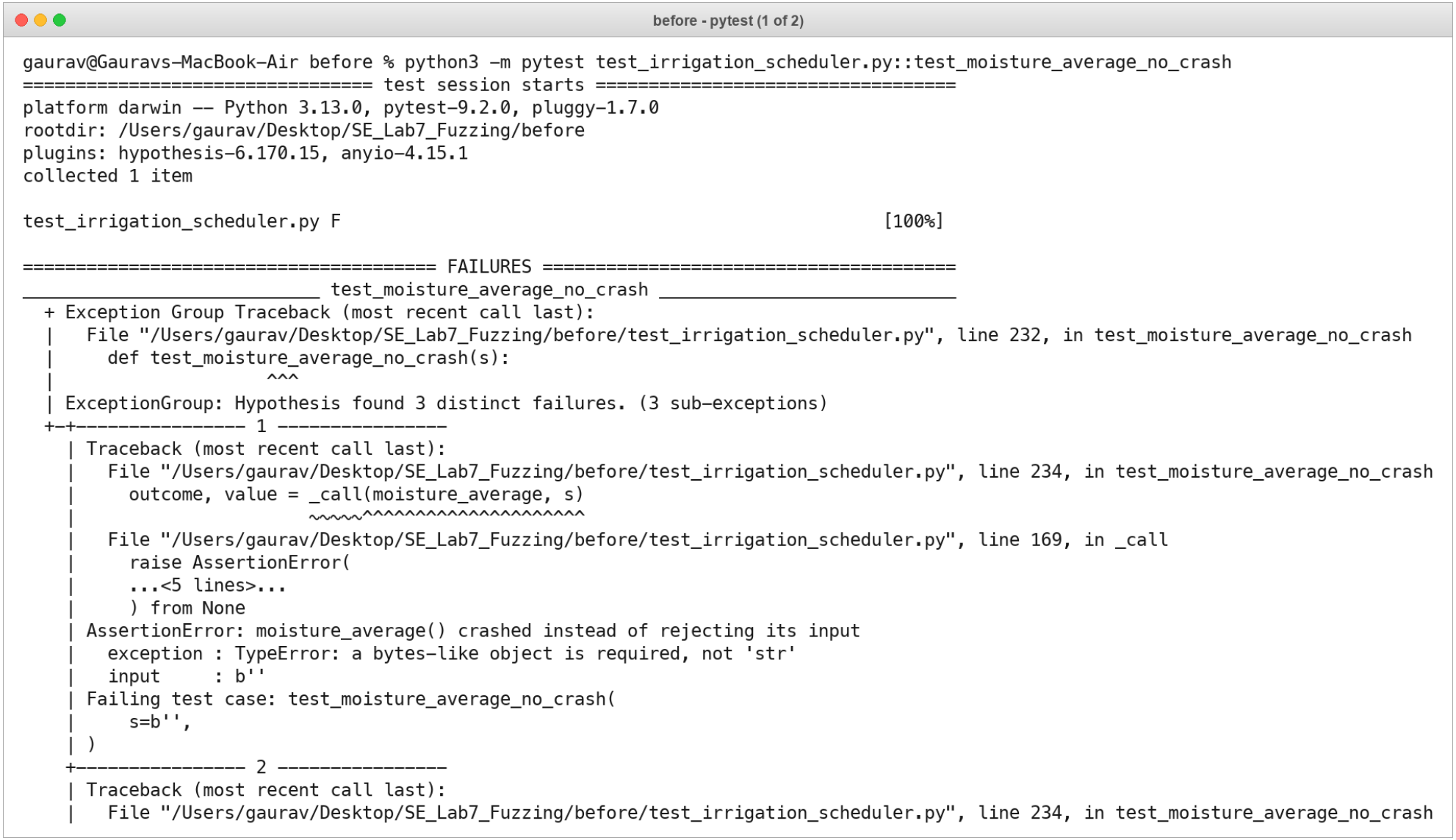


Figure 6.1 — Step 4: three distinct crashes in `moisture_average`.

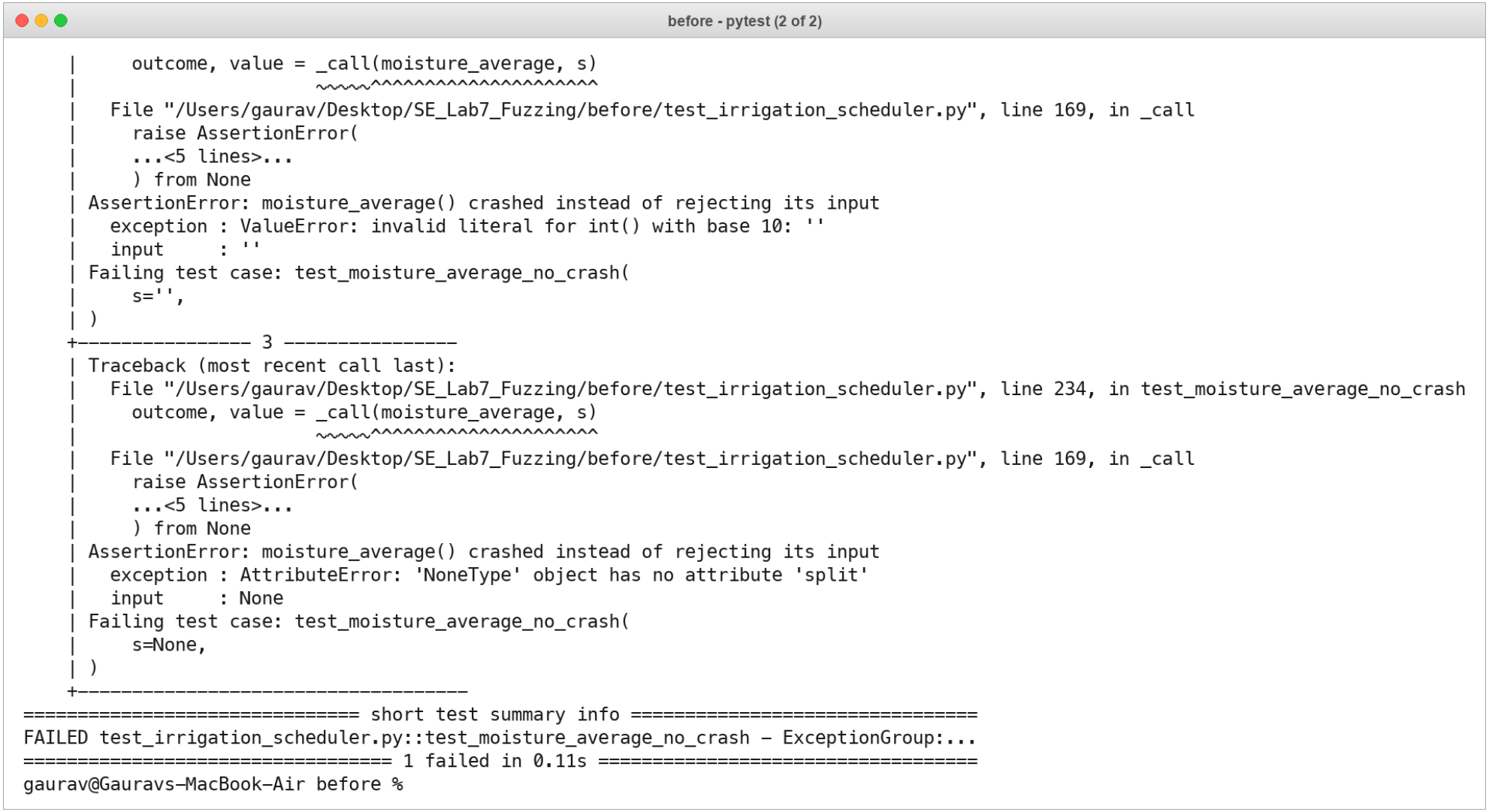


Figure 6.2 — Step 4: three distinct crashes in `moisture_average`.

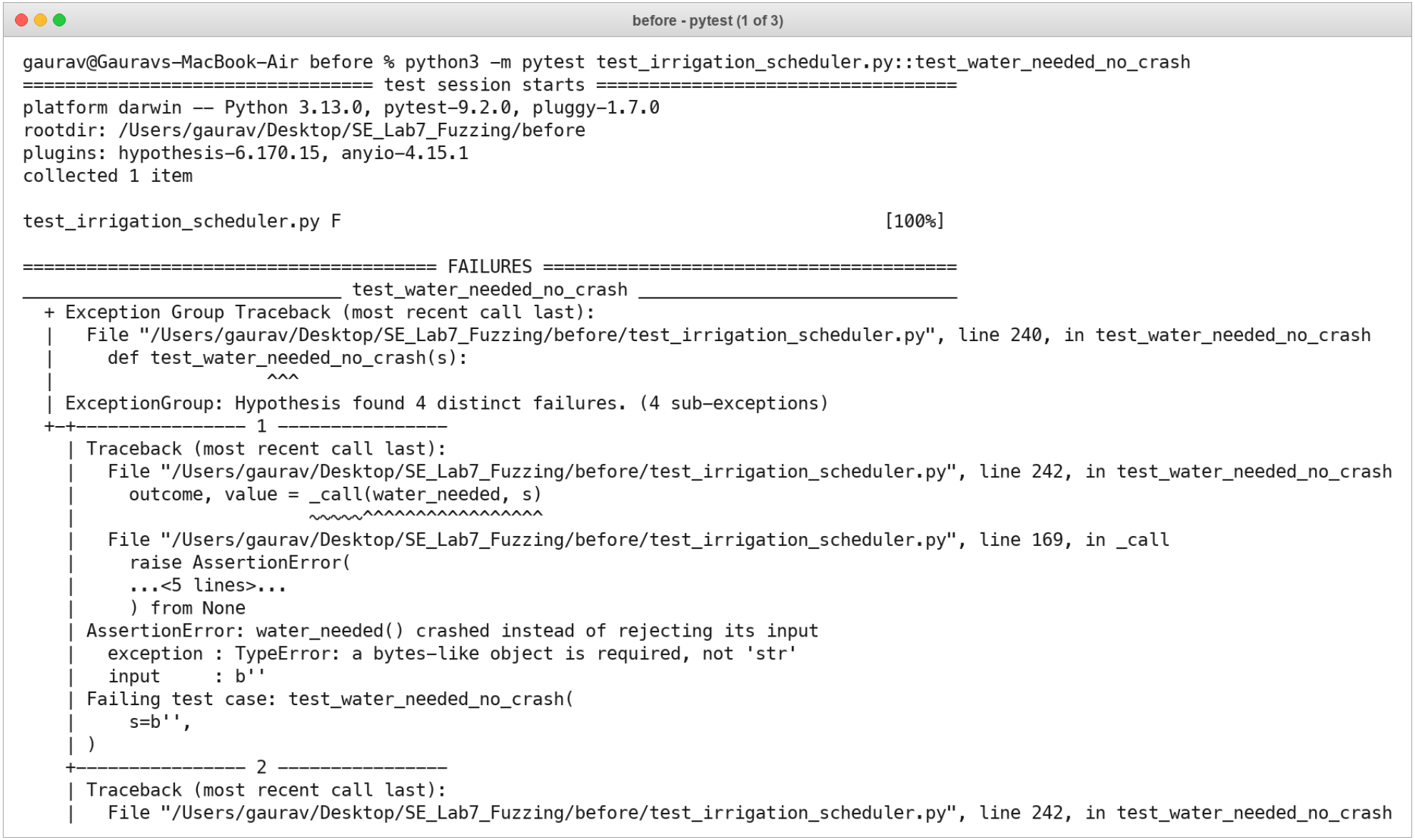


Figure 7.1 — Step 4: four distinct crashes in `water_needed`.

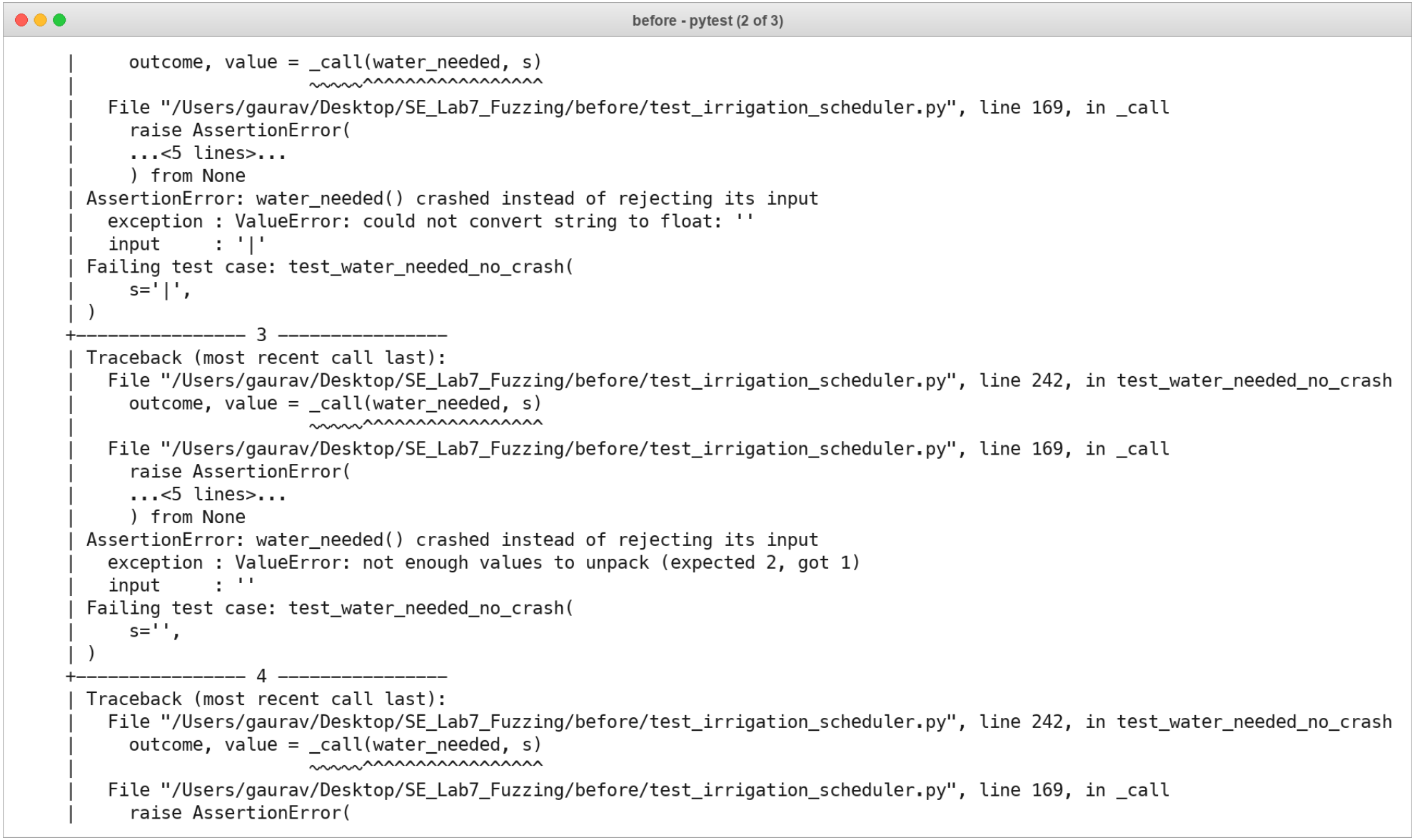


Figure 7.2 — Step 4: four distinct crashes in `water_needed`.

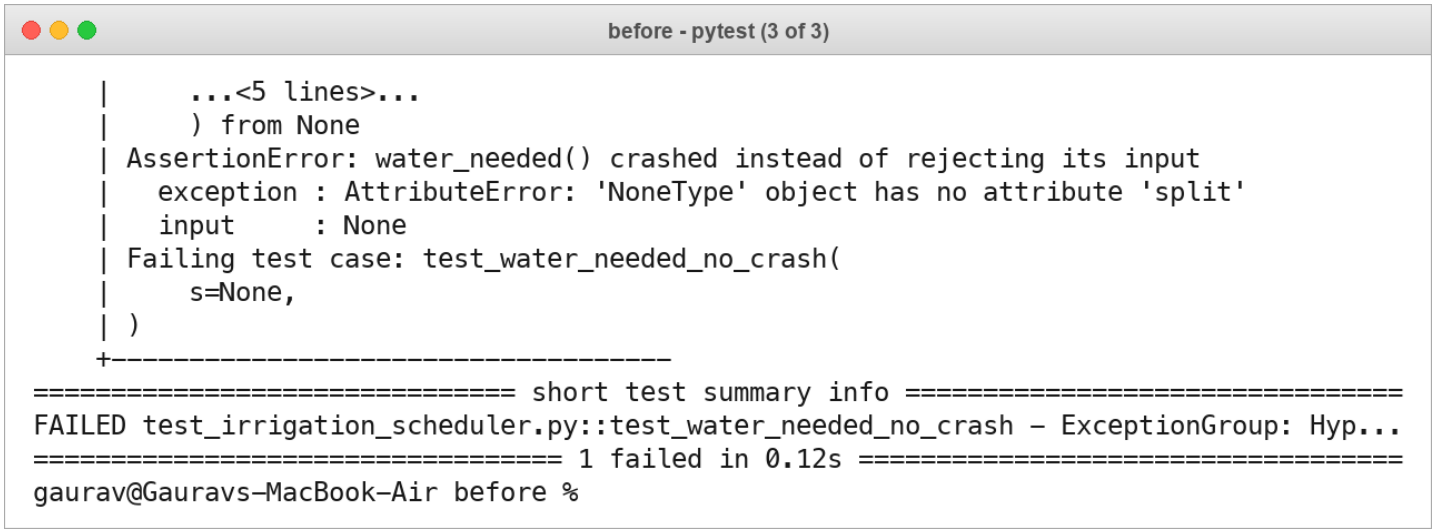


Figure 7.3 — Step 4: four distinct crashes in `water_needed`.

6.1 The defects the fuzzer found

Family A — uncontrolled crashes

#	Function	Minimal input	Exception that escaped
A1	all three	`None`	`AttributeError: 'NoneType' object has no attribute 'split'`
A2	all three	`b''`	`TypeError: a bytes-like object is required, not 'str'`
A3	parse_zone water_needed	`''`	`ValueError: not enough values to unpack (expected 2, got 1)`
A4	parse_zone	`':'`	`ValueError: could not convert string to float: ''`
A5	water_needed	`'|'`	`ValueError: could not convert string to float: ''`
A6	moisture_average	`'' , '1,,3'`	`ValueError: invalid literal for int() with base 10: ''`
A7	moisture_average	`'one,two'`	`ValueError: invalid literal for int() with base 10: 'one'`
A8	parse_zone water_needed	`'a:1:2' , '120|8|2'`	`ValueError: too many values to unpack (expected 2)`
A9	moisture_average	`'12.5'`	`ValueError: invalid literal for int() with base 10: '12.5'`

One candidate did **not** turn out to be a bug, and the fuzzer is what settled it. The `sum(values) // len(values)` in `moisture_average` looks like a `ZeroDivisionError` waiting to happen, but no input reaches it: `str.split` always returns at least one field, so `len(values)` is never zero. The division is safe only by accident of how `split` behaves, so the hardened version checks the list is non-empty anyway rather than relying on that accident.

Family B — nonsense accepted silently

These inputs produce no traceback at all. The program carries on and returns an irrigation figure that is physically meaningless, which in a real deployment is the worse of the two failure modes.

#	Input	What the buggy module returns	Why it is wrong
B1	`'north:-120'`	`('North', -120.0)`	A field of minus 120 square metres.
B2	`'north:0'`	`('North', 0.0)`	A zone with no area at all.
B3	`'north:nan'`	`('North', nan)`	`float()` accepts the literal 'nan'; the area is then not a number.
B4	`'north:inf'`	`('North', inf)`	An infinite field area.
B5	`'north:1e400'`	`('North', inf)`	Overflows silently to infinity.
B6	`'north:\u0664\u0663'`	`('North', 43.0)`	`float()` accepts Arabic-Indic digits, so a typo in another script parses as a number.
B7	`'-5'`	`-5`	Soil moisture of minus five percent.
B8	`'101'`	`101`	Soil moisture above 100 percent.
B9	`'99999999999999999999'`	`9999999999999999999`	A moisture percentage of 10^{20}.
B10	`'nan|8'`	`nan`	`max(nan, 0)` returns `nan`, so the comparison guard does nothing.
B11	`'1e400|8'`	`inf`	An infinite volume of water.
B12	`'-120|8'`	`0`	Negative area is accepted; the result is masked by `max(..., 0)`.

#	Input	What the buggy module returns	Why it is wrong
B13	`'10|500'`	`0` *(an int)*	The return type changes from `float` to `int` depending on the input — caught by `test_water_needed_is_never_negative`.
B14	`100,000 readings`	`computed in full`	Unbounded work for one line of input — a cheap denial of service.

7. Step 5 — Hardening the module

The module was rewritten to validate first and compute second. Three ideas do most of the work.

One documented error type. A new `InvalidInputError(ValueError)` is the only exception the public functions raise. Subclassing `ValueError` keeps any existing `except ValueError` handler working, while giving callers — and the test suite — a way to tell a deliberate rejection apart from a crash.

Stricter literals than the built-ins. Validation uses explicit ASCII-only regular expressions rather than letting `float()` and `int()` decide. Those built-ins accept `'nan'`, `'inf'`, `'1e400'`, `'1_000'` and non-ASCII digits, and every one of those was a bug in family B.

A stated domain. Each quantity is given the range it can physically take, so out-of-range values are refused instead of being carried into the schedule.

Quantity	Accepted range	Rejects
zone name	letters, digits, space, '-', '_'; must start with a letter; 1–50 characters	empty, blank, punctuation-only, over-long names
area	plain decimal, 0 < area ≤ 1,000,000 m²	zero, negative, nan, inf, exponent forms, non-ASCII digits
moisture reading	whole number, 0 ≤ r ≤ 100 percent; at most 100 readings	empty fields, decimals, negatives, values above 100, huge lists
rainfall	plain decimal, 0 ≤ r ≤ 1,000 mm	negative, nan, inf
any raw input	a str of at most 500 characters	None, bytes, int, list, dict; oversized strings

For valid input the arithmetic is untouched: `name.title()`, `sum(values) // len(values)` and `area * 5 - rainfall` are exactly as they were, which is what the Group 2 tests exist to guarantee. The one deliberate change is that `water_needed` now always returns a `float` instead of sometimes returning a bare `int` 0 (defect B13).

The command-line front end was hardened too. Each prompt now re-asks up to three times and prints the validation message instead of a traceback, and a closed input stream or a Ctrl-C ends the step cleanly. Figure 9 shows this: a missing separator, a negative area, a doubled comma and a doubled pipe are each reported and recovered from, and the whitespace-padded `' south field : 240.5 '` is accepted and title-cased.

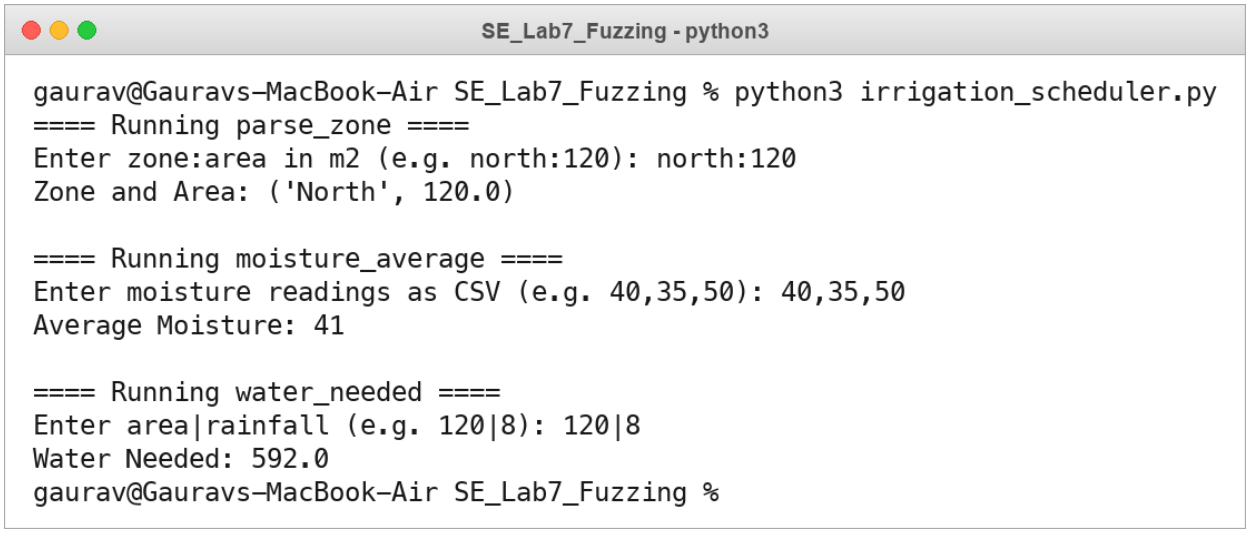


Figure 8 — Step 5: the hardened module gives exactly the same answers as before for well-formed input.

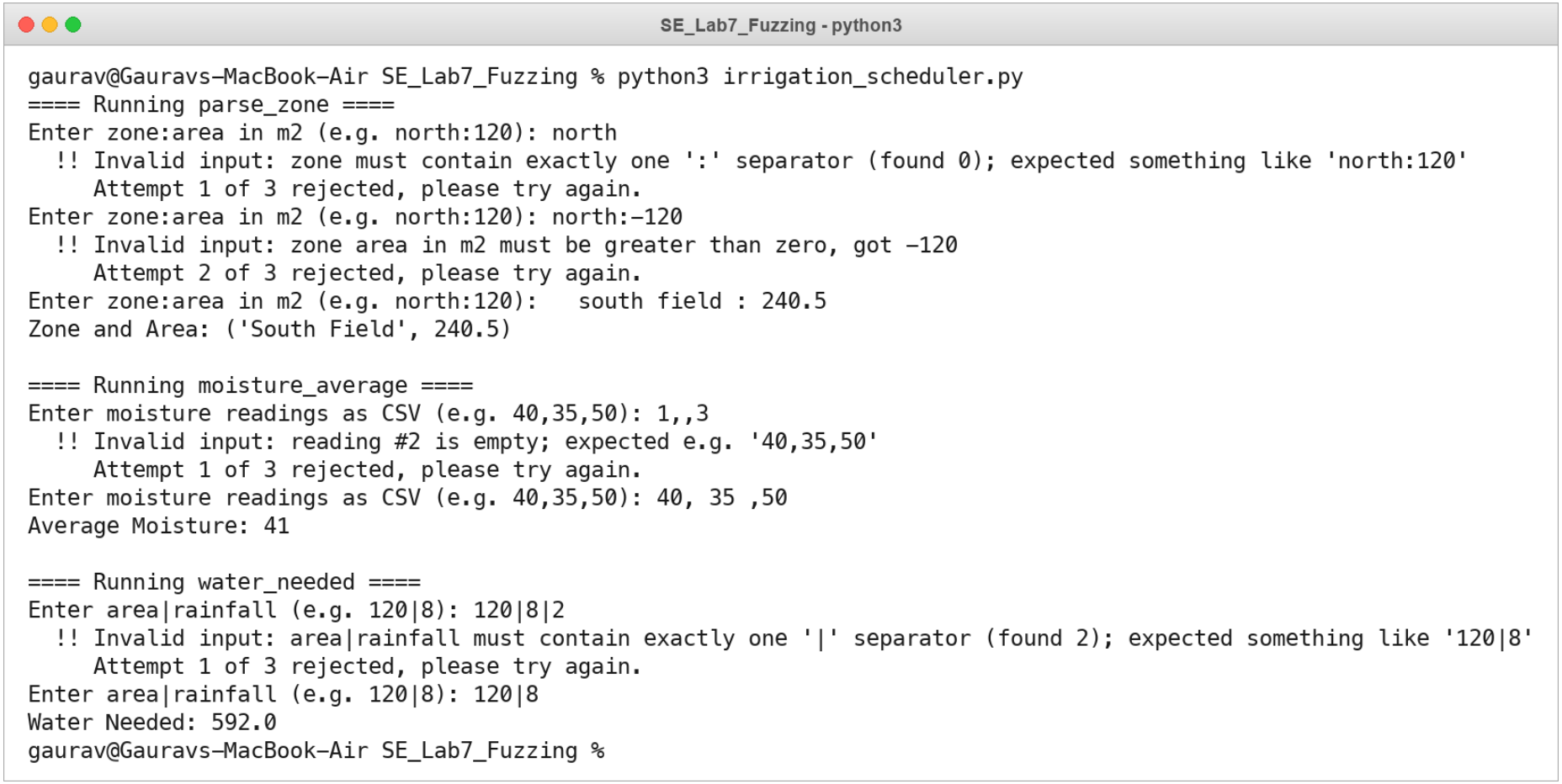


Figure 9 — Step 5: invalid input is now reported and recovered from instead of crashing.

8. Step 6 — Re-running the fuzz tests

The test file was not edited between Step 4 and Step 6 — it is the same 59 tests, byte for byte. Against the hardened module all 59 pass. Figure 10 lists them individually and Figure 11 is the summary line.

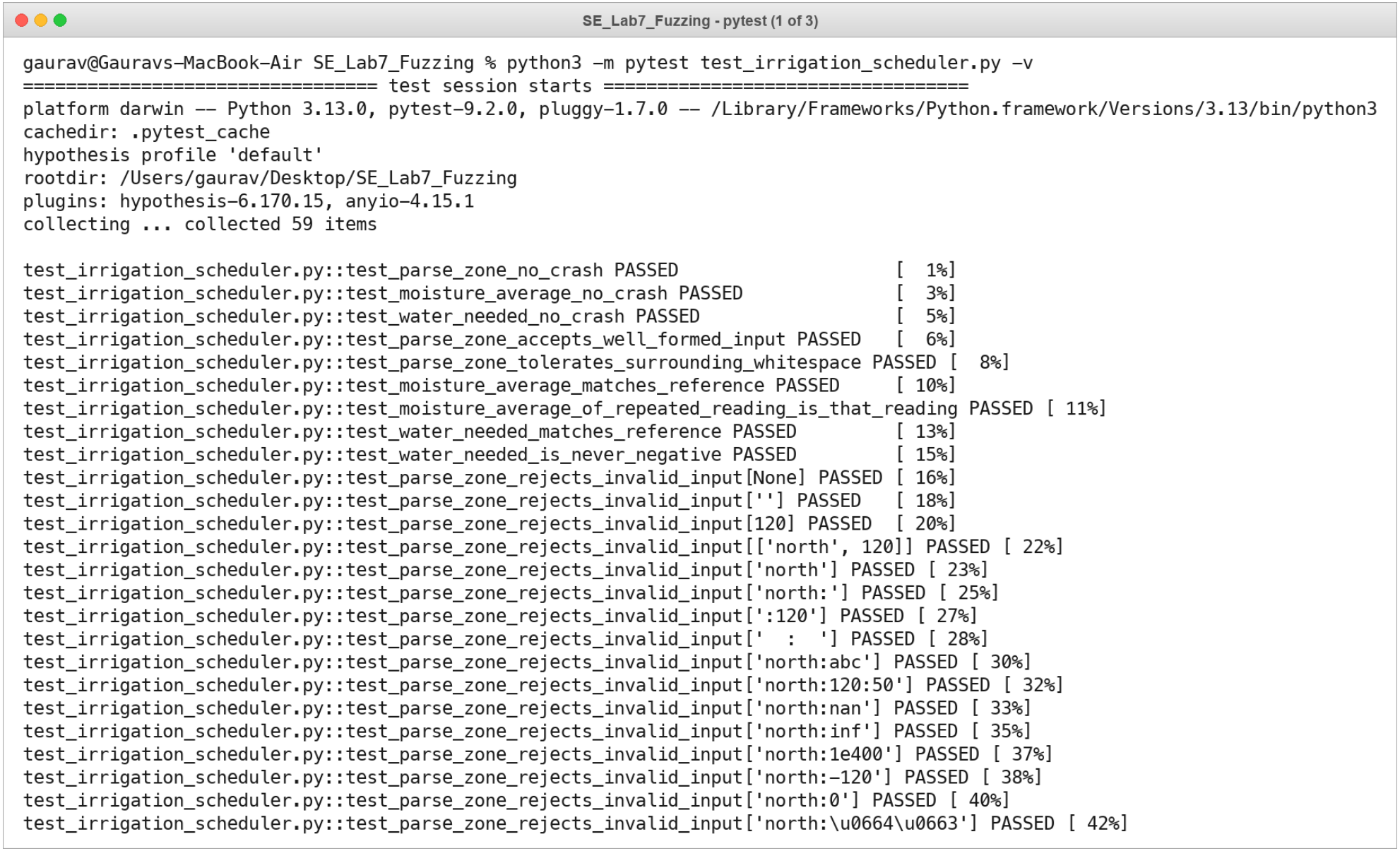


Figure 10.1 — Step 6: all 59 tests pass against the hardened module (verbose listing).



Figure 10.2 — Step 6: all 59 tests pass against the hardened module (verbose listing).

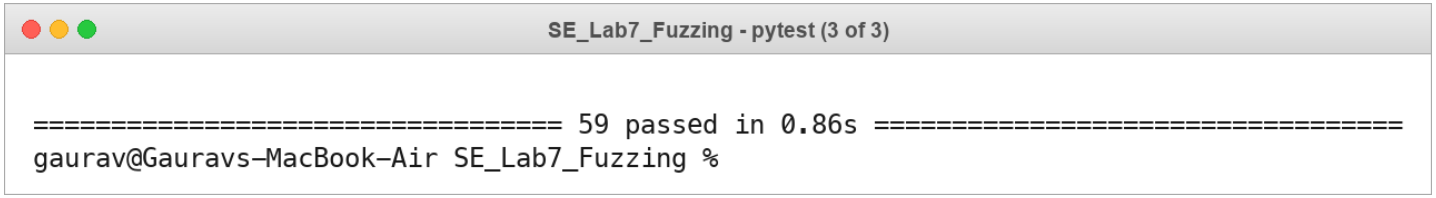


Figure 10.3 — Step 6: all 59 tests pass against the hardened module (verbose listing).

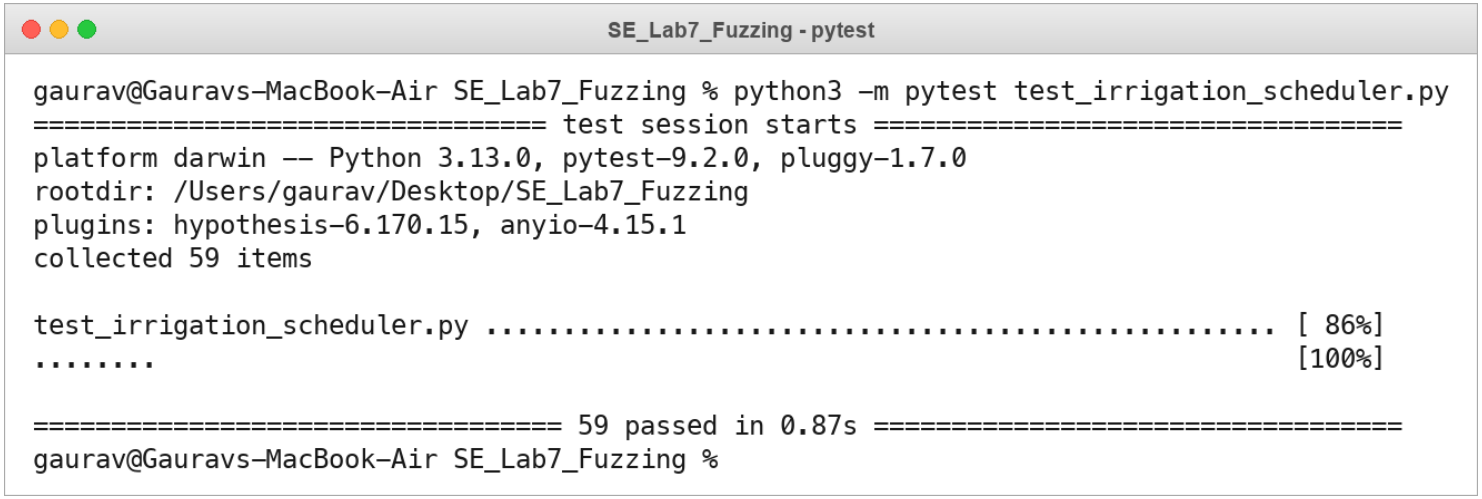


Figure 11 — Step 6: **59 passed** — the same suite that reported 54 failures before the fix.

8.1 An extended fuzz campaign

Two hundred examples per property is enough to find bugs this shallow, but not enough to be confident none is left. The three properties were therefore re-run with a wider strategy and 6,000 examples each — 18,000 generated inputs in total, including control characters, surrogate-free Unicode, nested separators, `'0x10'`, `'1e-400'` and thirty-digit integers. No input produced an unexpected exception and no returned value broke a post-condition:

```
18000 fuzz examples executed across 3 functions
violations found: 0
```

The six worked examples in the module's own docstrings were also checked with `python3 -m doctest irrigation_scheduler.py`, and all six pass, so the documentation and the code agree.

9. Conclusion

	Before Step 5	After Step 5
Fuzz tests passing	5 of 59	**59 of 59**
Distinct crashes Hypothesis reported	11 — 4 + 3 + 4 across the three functions	**0**
Nonsense values accepted	14 recorded (family B)	**0**
Behaviour on malformed input	traceback, process exits	message, prompt re-asked
Behaviour on valid input	correct	unchanged — still correct

What this exercise demonstrated:

Fuzzers find what you would not think to test. Of the eleven crashes reported in Figures 5 to 7, the minimal inputs were `None`, `''`, `b''` and `':'`. They are obvious in hindsight, which is exactly the point — they were not obvious enough for anyone to have written a test for them.

Shrinking is what makes a fuzzer usable. Hypothesis did not report the random 40-character string that first failed; it reported the one-character input `':'`. A minimal counter-example is a bug report that can be fixed in one sitting.

"It did not crash" is too weak a property. The quiet defects in family B — negative areas, moisture above 100 %, `nan` slipping through `max(need, 0)` — raise no exception at all. They were found only because the properties assert post-conditions on the value returned, not just the absence of an exception.

The built-in parsers are more permissive than they look. `float()` and `int()` accept `'nan'`, `'inf'`, `'1e400'`, underscores and non-ASCII digit characters. Defensive parsing means validating against an explicit pattern first, rather than treating "it converted without raising" as proof the input was sensible.

A fuzz test is a contract, so the contract has to be written down. Hardening the module was straightforward only because the accepted domain of every quantity was stated first. Validation code with no stated domain has nothing to be correct against.

10. Deliverables

File	Contents
`Lab7_Fuzz_Testing_Report_PES1UG24AM103.pdf`	This document, with the screenshots for Steps 2, 4, 5 and 6.
`irrigation_scheduler.py`	The fixed application script (Appendix A).

File	Contents
`test_irrigation_scheduler.py`	The completed Hypothesis test suite (Appendix B).
`irrigation_scheduler_BEFORE_fix.py`	The original buggy module, kept for reference.
`pytest.ini`	Keeps `before/` out of collection, so `pytest` run in this folder tests the fixed module.
`before/`	The original module plus the same test file, so the Step 2 and Step 4 runs can be reproduced.
`screenshots/`	The screenshot images used in this report.
`transcripts/`	The plain-text terminal transcripts the screenshots were rendered from.

Appendix A — `irrigation_scheduler.py`

The hardened application script, as submitted.

```
# ================================================================================
#  Lab 7 - Fuzzing for Input Validation Bugs
#  Name  : Gaurav Girish
#  SRN   : PES1UG24AM103
#  Repo  : L6_L7_37_irrigation_scheduler
#  File  : irrigation_scheduler.py   (hardened - Step 5)
# ================================================================================
"""Irrigation scheduler: parse field zones, average soil moisture, estimate water.

WHAT CHANGED AND WHY
--------------------
The original version trusted its input completely. It called ``str.split`` on
whatever it was handed, unpacked the result into a fixed number of names, and
fed the pieces straight to ``float()`` and ``int()``. Fuzzing the three
functions with Hypothesis exposed two separate families of defect:

1. CRASHES - an unhandled exception escapes to the caller
     ``AttributeError``     ``parse_zone(None)``          - None has no .split
     ``TypeError``          ``parse_zone(b'')``           - bytes vs str
     ``ValueError``         ``parse_zone('')``            - not enough values
     ``ValueError``         ``parse_zone('north:abc')``   - bad float literal
     ``ValueError``         ``parse_zone('a:1:2')``       - too many values
     ``ValueError``         ``moisture_average('1,,3')``  - bad int literal
     ``ValueError``         ``moisture_average('')``      - bad int literal

   The ``// len(values)`` in ``moisture_average`` looks like a
   ``ZeroDivisionError`` waiting to happen, and fuzzing settled the question:
   it is unreachable, because ``str.split`` always returns at least one field.
   The division is safe only by accident, so the hardened version checks the
   list is non-empty anyway rather than relying on that accident.

2. SILENT ACCEPTANCE OF NONSENSE - no crash, but the answer is meaningless
     ``parse_zone('north:-120')``   -> an area of minus 120 square metres
     ``parse_zone('north:nan')``    -> float() accepts 'nan'
     ``parse_zone('north:1e400')``  -> overflows to an infinite area
     ``parse_zone('north:\u0664\u0663')``  -> non-ASCII digits accepted quietly
     ``moisture_average('-5')``     -> soil moisture of minus five percent
     ``moisture_average('101')``    -> moisture above 100 percent
     ``water_needed('nan|8')``      -> NaN slips through ``max(need, 0)``
     a 100 000-element reading list -> unbounded work for one line of input

THE CONTRACT NOW GUARANTEED
---------------------------
Every public function either returns a result that satisfies its documented
post-conditions, or raises ``InvalidInputError`` with a message a user can act
on. No other exception can escape. Validation is done up front (fail fast),
the accepted domain is stated explicitly, and the arithmetic for valid input is
byte-for-byte the behaviour of the original code.

ACCEPTED DOMAIN
---------------
  zone name         letters / digits / space / '-' / '_', must start with a
                    letter, 1..50 characters, surrounding spaces ignored
  area              plain decimal metres squared, 0 < area <= 1 000 000
  moisture reading  whole percentage, 0 <= reading <= 100, at most 100 readings
  rainfall          plain decimal millimetres, 0 <= rainfall <= 1000
  any raw input     a str of at most 500 characters

Exponent forms ('1e400'), the special literals 'nan' / 'inf', thousands
underscores ('1_000') and non-ASCII digit characters are deliberately refused:
``float()`` and ``int()`` all accept them, and every one of them was a bug.
"""

import math
import re

# --- limits of the accepted domain ------------------------------------------
MAX_INPUT_LEN = 500          # characters of raw input
```

```
MAX_NAME_LEN = 50            # characters of a zone name
MAX_AREA_M2 = 1_000_000.0    # one square kilometre
MAX_RAINFALL_MM = 1_000.0
MIN_MOISTURE = 0             # percent
MAX_MOISTURE = 100           # percent
MAX_READINGS = 100           # readings in one CSV line
LITRES_PER_M2 = 5            # water requirement factor (unchanged)

# --- strict ASCII-only literal patterns ----------------------------------
# These are deliberately stricter than float() and int(). Both built-ins accept
# 'nan', 'inf', '1e400', '1_000' and non-ASCII digits such as '\u0664\u0663',
# and every one of those was a bug found by fuzzing.
_DECIMAL_RE = re.compile(r"[+-]?(?:[0-9]+(?:\.[0-9]*)?|\.[0-9]+)")
_INTEGER_RE = re.compile(r"[+-]?[0-9]+")
_NAME_RE = re.compile(r"[A-Za-z][A-Za-z0-9 _-]*")


class InvalidInputError(ValueError):
    """Raised when input fails validation.

    This is the only exception the public functions of this module raise, which
    is what lets a caller - or a fuzz test - tell a deliberate rejection apart
    from a crash. It subclasses ``ValueError`` so existing ``except ValueError``
    handlers keep working.
    """


# ---------------------------------------------------------------------------
# Internal validation helpers
# ---------------------------------------------------------------------------
def _require_text(value, field):
    """Return ``value`` if it is a string of a sane length, else reject it."""
    if not isinstance(value, str):
        raise InvalidInputError(
            f"{field} must be given as text, got {type(value).__name__}"
        )
    if len(value) > MAX_INPUT_LEN:
        raise InvalidInputError(
            f"{field} is too long ({len(value)} characters, limit {MAX_INPUT_LEN})"
        )
    return value


def _split_once(text, separator, field, example):
    """Split ``text`` on ``separator``, insisting on exactly one separator."""
    parts = text.split(separator)
    if len(parts) != 2:
        found = len(parts) - 1
        raise InvalidInputError(
            f"{field} must contain exactly one '{separator}' separator "
            f"(found {found}); expected something like {example!r}"
        )
    return parts[0], parts[1]


def _to_decimal(raw, field, maximum, allow_zero):
    """Parse ``raw`` as a bounded, finite decimal number."""
    token = raw.strip()
    if not token:
        raise InvalidInputError(f"{field} is missing")
    if not _DECIMAL_RE.fullmatch(token):
        raise InvalidInputError(
            f"{field} must be a plain decimal number such as '120' or '12.5', "
            f"got {raw!r}"
        )
    value = float(token)
    if not math.isfinite(value):
        raise InvalidInputError(f"{field} must be a finite number, got {raw!r}")
    lower_ok = value >= 0 if allow_zero else value > 0
    if not lower_ok:
        wording = "zero or greater" if allow_zero else "greater than zero"
        raise InvalidInputError(f"{field} must be {wording}, got {value:g}")
    if value > maximum:
```

```
        raise InvalidInputError(f"{field} must not exceed {maximum:g}, got {value:g}")
    return value


# ---------------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------------
def parse_zone(text):
    """Parse ``'<name>:<area>'`` into a ``(title-cased name, area)`` pair.

    >>> parse_zone("north:120")
    ('North', 120.0)
    >>> parse_zone("  south field : 240.5 ")
    ('South Field', 240.5)

    Returns a 2-tuple of a non-empty ``str`` and a finite ``float`` greater
    than zero. Raises ``InvalidInputError`` for anything else.
    """
    text = _require_text(text, "zone")
    name_part, area_part = _split_once(text, ":", "zone", "north:120")

    name = name_part.strip()
    if not name:
        raise InvalidInputError("zone name is missing; expected e.g. 'north:120'")
    if len(name) > MAX_NAME_LEN:
        raise InvalidInputError(
            f"zone name is too long ({len(name)} characters, limit {MAX_NAME_LEN})"
        )
    if not _NAME_RE.fullmatch(name):
        raise InvalidInputError(
            "zone name must start with a letter and may only contain letters, "
            f"digits, spaces, '-' and '_', got {name_part!r}"
        )

    area = _to_decimal(area_part, "zone area in m2", MAX_AREA_M2, allow_zero=False)
    return name.title(), area


def moisture_average(readings_csv):
    """Average a comma-separated list of soil-moisture percentages.

    >>> moisture_average("40,35,50")
    41
    >>> moisture_average(" 40 , 35 ")
    37

    Returns an ``int`` between 0 and 100 (floor division, as before). Raises
    ``InvalidInputError`` if the list is empty, over-long, or holds anything
    that is not a whole percentage in range.
    """
    readings_csv = _require_text(readings_csv, "moisture readings")

    fields = readings_csv.split(",")
    if len(fields) > MAX_READINGS:
        raise InvalidInputError(
            f"too many readings ({len(fields)}, limit {MAX_READINGS})"
        )

    values = []
    for position, field in enumerate(fields, start=1):
        token = field.strip()
        if not token:
            raise InvalidInputError(
                f"reading #{position} is empty; expected e.g. '40,35,50'"
            )
        if not _INTEGER_RE.fullmatch(token):
            raise InvalidInputError(
                f"reading #{position} must be a whole number, got {field!r}"
            )
        value = int(token)
        if not MIN_MOISTURE <= value <= MAX_MOISTURE:
            raise InvalidInputError(
                f"reading #{position} must be a percentage between "
```

```
                f"{MIN_MOISTURE} and {MAX_MOISTURE}, got {value}"
            )
        values.append(value)

    if not values:  # defensive: unreachable, str.split always yields one field
        raise InvalidInputError("at least one moisture reading is required")

    return sum(values) // len(values)


def water_needed(text):
    """Estimate water required from ``'<area>|<rainfall>'``.

    >>> water_needed("120|8")
    592.0
    >>> water_needed("10|500")
    0.0

    Returns a finite ``float`` of zero or more - the original returned a bare
    ``int`` 0 in the "no water needed" case, so the return type is now
    consistent. Raises ``InvalidInputError`` for anything outside the domain.
    """
    text = _require_text(text, "area|rainfall")
    area_part, rain_part = _split_once(text, "|", "area|rainfall", "120|8")

    area = _to_decimal(area_part, "plot area in m2", MAX_AREA_M2, allow_zero=False)
    rainfall = _to_decimal(
        rain_part, "rainfall in mm", MAX_RAINFALL_MM, allow_zero=True
    )

    need = area * LITRES_PER_M2 - rainfall
    return float(max(need, 0.0))


# ---------------------------------------------------------------------------
# Command-line front end
# ---------------------------------------------------------------------------
MAX_ATTEMPTS = 3


def ask(prompt, parser):
    """Prompt until ``parser`` accepts the answer, then return its result.

    Gives up after ``MAX_ATTEMPTS`` tries and returns ``None``. A closed input
    stream (Ctrl-D) or Ctrl-C ends the step cleanly instead of dumping a
    traceback - the whole point of the hardening.
    """
    for attempt in range(1, MAX_ATTEMPTS + 1):
        try:
            raw = input(prompt)
        except (EOFError, KeyboardInterrupt):
            print("\n  -- input stopped, skipping this step --")
            return None
        try:
            return parser(raw)
        except InvalidInputError as exc:
            print(f"  !! Invalid input: {exc}")
            if attempt < MAX_ATTEMPTS:
                print(
                    f"     Attempt {attempt} of {MAX_ATTEMPTS} rejected, "
                    "please try again."
                )
    print(f"  !! {MAX_ATTEMPTS} invalid attempts, skipping this step.")
    return None


def main():
    print("==== Running parse_zone ====")
    result = ask("Enter zone:area in m2 (e.g. north:120): ", parse_zone)
    if result is not None:
        print("Zone and Area:", result)

    print("\n==== Running moisture_average ====")
```

```
    result = ask("Enter moisture readings as CSV (e.g. 40,35,50): ", moisture_average)
    if result is not None:
        print("Average Moisture:", result)

    print("\n==== Running water_needed ====")
    result = ask("Enter area|rainfall (e.g. 120|8): ", water_needed)
    if result is not None:
        print("Water Needed:", result)


if __name__ == "__main__":
    main()
```

Appendix B — `test_irrigation_scheduler.py`

The completed Hypothesis test suite, as submitted. This file is identical in Step 4 and Step 6.

```
# ================================================================================
#  Lab 7 - Fuzzing for Input Validation Bugs
#  Name  : Gaurav Girish
#  SRN   : PES1UG24AM103
#  Repo  : L6_L7_37_irrigation_scheduler
#  File  : test_irrigation_scheduler.py   (completed - Step 3)
# ================================================================================
"""Property-based (fuzz) test suite for ``irrigation_scheduler`` using Hypothesis.

THE CONTRACT BEING TESTED
-------------------------
For *every* input Hypothesis is able to invent, each public function must do
exactly one of two things:

  (a) ACCEPT  the input and return a result that satisfies the function's
              documented post-conditions, or
  (b) REJECT  the input by raising ``InvalidInputError`` - the single,
              documented validation error of the module.

Anything else is an uncontrolled crash and therefore a bug: a ``ValueError``
escaping from ``float()`` or ``int()``, an ``AttributeError`` from ``None``, a
``TypeError`` from ``bytes``, an ``IndexError``, an ``OverflowError`` - these
are the failures a fuzzer is built to find.

Silently accepting nonsense is treated as a bug too. A soil-moisture reading
of ``-999``, a field area of ``-120 m2`` or an area of ``inf`` must not sail
through unnoticed, so the post-condition checks below assert domain ranges as
well as types.

DOCUMENTED DOMAIN (quantities this module is about)
---------------------------------------------------
  * zone name        - letters / digits / space / '-' / '_', starts with a
                       letter, 1..50 characters
  * area             - a finite number of square metres, 0 < area <= 1e6
  * moisture reading - a whole percentage, 0 <= reading <= 100
  * rainfall         - finite millimetres, 0 <= rainfall <= 1000

HOW THIS FILE BEHAVES BEFORE AND AFTER THE FIX
----------------------------------------------
``InvalidInputError`` does not exist in the original buggy module, so the
import below falls back to a local placeholder class. With the placeholder in
place *nothing* counts as a deliberate rejection, which is precisely correct
for the buggy version: every malformed input shows up as a crash and the suite
fails. After Step 5 the real class is imported from the module and the same
unchanged suite passes.

Run with:  pytest test_irrigation_scheduler.py
"""

import math

import pytest
from hypothesis import HealthCheck, given, settings, strategies as st

from irrigation_scheduler import moisture_average, parse_zone, water_needed

try:  # the hardened module exports its own validation error ...
    from irrigation_scheduler import InvalidInputError
except ImportError:  # ... the original buggy module does not.

    class InvalidInputError(ValueError):
        """Placeholder used while testing the un-fixed module.

        Nothing in the buggy code raises this, so every rejected input is
        correctly reported as an uncontrolled crash.
        """
```

```
# ----------------------------------------------------------------------------
# Hypothesis configuration
# ----------------------------------------------------------------------------
# deadline=None: a slow example is not the kind of bug this lab is hunting,
# and a DeadlineExceeded would be misreported as a crash by the helper below.
settings.register_profile(
    "lab7",
    max_examples=200,
    deadline=None,
    suppress_health_check=[HealthCheck.too_slow, HealthCheck.filter_too_much],
)
settings.load_profile("lab7")


# ----------------------------------------------------------------------------
# Fuzzing strategies
# ----------------------------------------------------------------------------
# 1. Unstructured fuzz - "throw literally anything at it", including the wrong
#    type altogether. This is the st.text() | st.none() idea from the handout,
#    widened so that type confusion is covered as well.
WILD = (
    st.text()
    | st.none()
    | st.integers()
    | st.floats()
    | st.booleans()
    | st.binary()
    | st.lists(st.integers())
    | st.dictionaries(st.text(), st.integers())
)

# 2. Structure-aware fuzz. Purely random text almost never contains a ':' or a
#    '|', so unstructured fuzzing alone keeps tripping over the very first
#    split() and never reaches the parsing code behind it. These pieces are
#    glued together with real separators so the fuzzer gets *past* the split
#    and can attack float()/int() and the arithmetic underneath.
PIECES = st.sampled_from(
    [
        "",            # empty field
        " ",           # whitespace-only field
        "north",       # a name where a number belongs
        "North West",  # name with a space
        "abc",
        "120",
        "0",           # zero area  -> a zone with no area
        "-5",          # negative   -> impossible area / moisture
        "12.5",        # float where an int belongs
        "101",         # moisture above 100%
        "nan",         # float() happily accepts these three ...
        "inf",
        "1e400",       # ... and this one silently becomes +inf
        "\u0664\u0663",  # Arabic-Indic digits: float()/int() accept them
        "\uff11\uff12",  # full-width digits: accepted too
        "+7",
        ".",
        "-",
        "1_000",       # underscore separator
        "99999999999999999999999999",
        " 120 ",
        "None",
    ]
)
STRUCTURED = st.builds(
    lambda parts, sep: sep.join(parts),
    st.lists(PIECES, min_size=1, max_size=3),
    st.sampled_from([":", "|", ",", ";", " ", ""]),
)

FUZZ = WILD | STRUCTURED

# 3. Well-formed inputs, generated from a regex, for the "it must still do the
#    right thing" properties.
VALID_NAME = st.from_regex(r"\A[A-Za-z][A-Za-z0-9 _-]{0,19}\Z", fullmatch=True)
```

```
VALID_AREA_STR = st.from_regex(r"\A0*[1-9][0-9]{0,5}(\.[0-9]{1,3})?\Z", fullmatch=True)
VALID_RAIN_STR = st.from_regex(r"\A(0|[1-9][0-9]{0,2})(\.[0-9]{1,2})?\Z", fullmatch=True)
PADDING = st.sampled_from(["", " ", "   ", "\t"])


# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def _short(value, limit=48):
    """``repr`` of ``value``, shortened so failure reports stay readable."""
    text = repr(value)
    return text if len(text) <= limit else text[: limit - 3] + "..."


def _call(func, raw):
    """Call ``func(raw)`` and classify what happened.

    Returns ``("ok", value)`` when the input was accepted and
    ``("rejected", exc)`` when it was deliberately refused.
    Raises ``AssertionError`` - i.e. fails the test - on any other exception,
    because that is an uncontrolled crash.
    """
    try:
        return "ok", func(raw)
    except InvalidInputError as exc:
        return "rejected", exc
    except Exception as exc:  # noqa: BLE001 - this broad catch *is* the detector
        raise AssertionError(
            "{}() crashed instead of rejecting its input\n"
            "  exception : {}: {}\n"
            "  input     : {}".format(
                func.__name__, type(exc).__name__, str(exc)[:60], _short(raw)
            )
        ) from None


def _fail(func_name, problem, raw, got):
    raise AssertionError(
        "{}() returned a result that breaks its contract\n"
        "  problem   : {}\n"
        "  returned  : {}\n"
        "  input     : {}".format(func_name, problem, _short(got), _short(raw))
    )


def _check_zone(raw, result):
    """Post-conditions for ``parse_zone``: (non-blank str, finite float > 0)."""
    if not (isinstance(result, tuple) and len(result) == 2):
        _fail("parse_zone", "not a (name, area) 2-tuple", raw, result)
    name, area = result
    if not isinstance(name, str) or not name.strip():
        _fail("parse_zone", "zone name is not a non-blank string", raw, result)
    if not isinstance(area, float):
        _fail("parse_zone", "area is not a float", raw, result)
    if not math.isfinite(area):
        _fail("parse_zone", "area is not a finite number", raw, result)
    if area <= 0:
        _fail("parse_zone", "area is not greater than zero", raw, result)


def _check_moisture(raw, average):
    """Post-conditions for ``moisture_average``: an int in 0..100."""
    if not isinstance(average, int) or isinstance(average, bool):
        _fail("moisture_average", "average is not an int", raw, average)
    if not 0 <= average <= 100:
        _fail("moisture_average", "average is outside 0-100 percent", raw, average)


def _check_water(raw, need):
    """Post-conditions for ``water_needed``: a finite float >= 0."""
    if not isinstance(need, float):
        _fail("water_needed", "water need is not a float", raw, need)
    if not math.isfinite(need):
```

```
        _fail("water_needed", "water need is not a finite number", raw, need)
    if need < 0:
        _fail("water_needed", "water need is negative", raw, need)


# ============================================================================
# 1. The three "no crash" properties asked for by the handout
# ============================================================================
@given(FUZZ)
def test_parse_zone_no_crash(s):
    """parse_zone must either parse the zone or reject it politely."""
    outcome, value = _call(parse_zone, s)
    if outcome == "ok":
        _check_zone(s, value)


@given(FUZZ)
def test_moisture_average_no_crash(s):
    """moisture_average must either average the readings or reject them."""
    outcome, value = _call(moisture_average, s)
    if outcome == "ok":
        _check_moisture(s, value)


@given(FUZZ)
def test_water_needed_no_crash(s):
    """water_needed must either compute a water volume or reject the input."""
    outcome, value = _call(water_needed, s)
    if outcome == "ok":
        _check_water(s, value)


# ============================================================================
# 2. Hardening must not change the behaviour of valid input
# ============================================================================
@given(VALID_NAME, VALID_AREA_STR)
def test_parse_zone_accepts_well_formed_input(name, area_str):
    zone, area = parse_zone(f"{name}:{area_str}")
    assert zone == name.strip().title()
    assert area == pytest.approx(float(area_str))


@given(VALID_NAME, VALID_AREA_STR, PADDING)
def test_parse_zone_tolerates_surrounding_whitespace(name, area_str, pad):
    expected = parse_zone(f"{name}:{area_str}")
    padded = f"{pad}{name}{pad}:{pad}{area_str}{pad}"
    assert parse_zone(padded) == expected


@given(st.lists(st.integers(min_value=0, max_value=100), min_size=1, max_size=20))
def test_moisture_average_matches_reference(values):
    readings_csv = ",".join(str(v) for v in values)
    assert moisture_average(readings_csv) == sum(values) // len(values)


@given(st.integers(min_value=0, max_value=100), st.integers(min_value=1, max_value=12))
def test_moisture_average_of_repeated_reading_is_that_reading(value, count):
    assert moisture_average(",".join([str(value)] * count)) == value


@given(VALID_AREA_STR, VALID_RAIN_STR)
def test_water_needed_matches_reference(area_str, rain_str):
    expected = max(float(area_str) * 5 - float(rain_str), 0.0)
    assert water_needed(f"{area_str}|{rain_str}") == pytest.approx(expected)


@given(VALID_AREA_STR, VALID_RAIN_STR)
def test_water_needed_is_never_negative(area_str, rain_str):
    _check_water(f"{area_str}|{rain_str}", water_needed(f"{area_str}|{rain_str}"))


# ============================================================================
# 3. Regression tests - the concrete inputs the fuzzer shrank down to.
```

```
#    Every one of these must be a clean rejection, not a crash and not a
#    silently wrong answer.
# ============================================================================
PARSE_ZONE_BAD = [
    None,                       # TypeError: 'NoneType' has no attribute 'split'
    "",                         # ValueError: not enough values to unpack
    120,                        # wrong type entirely
    ["north", 120],             # wrong type entirely
    "north",                    # separator missing
    "north:",                   # area missing
    ":120",                     # name missing
    "  :  ",                    # both blank
    "north:abc",                # ValueError from float()
    "north:120:50",             # too many separators
    "north:nan",                # float() accepts 'nan' -> silent NaN area
    "north:inf",                # float() accepts 'inf' -> silent infinite area
    "north:1e400",              # overflows to +inf
    "north:-120",               # negative area accepted silently
    "north:0",                  # a zone with no area at all
    "north:\u0664\u0663",       # non-ASCII digits accepted silently
    "4\u0663:120",              # name that is not a name
]

MOISTURE_BAD = [
    None,
    "",                         # ValueError: invalid literal for int()
    40,
    [40, 35],
    "1,,3",                     # empty element
    "one,two",                  # ValueError from int()
    "40, ,50",                  # whitespace-only element
    ",,",                       # two empty elements
    "40,35,",                   # trailing comma
    "40;35",                    # wrong separator
    "12.5",                     # float where an int is required
    "-5",                       # negative moisture accepted silently
    "101",                      # above 100% accepted silently
    "\u0664\u0663",             # non-ASCII digits accepted silently
    "99999999999999999999",     # absurd magnitude accepted silently
]

WATER_BAD = [
    None,
    "",
    120,
    ("120", "8"),
    "120",                      # separator missing
    "120|8|2",                  # too many separators
    "|",                        # both parts blank
    "120|",                     # rainfall missing
    "|8",                       # area missing
    "abc|8",                    # ValueError from float()
    "120|abc",
    "-120|8",                   # negative area accepted silently
    "120|-8",                   # negative rainfall accepted silently
    "0|8",                      # zero area
    "1e400|8",                  # +inf area -> inf litres of water
    "nan|8",                    # NaN propagates through max()
]


@pytest.mark.parametrize("bad", PARSE_ZONE_BAD, ids=repr)
def test_parse_zone_rejects_invalid_input(bad):
    with pytest.raises(InvalidInputError):
        parse_zone(bad)


@pytest.mark.parametrize("bad", MOISTURE_BAD, ids=repr)
def test_moisture_average_rejects_invalid_input(bad):
    with pytest.raises(InvalidInputError):
        moisture_average(bad)
```

```
@pytest.mark.parametrize("bad", WATER_BAD, ids=repr)
def test_water_needed_rejects_invalid_input(bad):
    with pytest.raises(InvalidInputError):
        water_needed(bad)


# ============================================================================
# 4. Robustness against oversized input (a cheap denial-of-service)
# ============================================================================
def test_oversized_reading_list_is_rejected():
    with pytest.raises(InvalidInputError):
        moisture_average(",".join(["50"] * 100_000))


def test_oversized_zone_string_is_rejected():
    with pytest.raises(InvalidInputError):
        parse_zone("n" * 100_000 + ":120")
```